



UNIVERSIDAD

Nicolás Felipe Bernal Gallo

Juan Daniel Bogotá Fuetes

Desarrollo Orientada a objetos
DOPO LAB

Laboratorio #3 Herencias e interfaces objetos
02/10/2025

PROFESOR: María Irma Díaz Roza

DESARROLLO ORIENTADO POR OBJETOS [DOPO-POOB]

Herencia e interfaces

ADEMAS Java desde consola

2025-2

Laboratorio 3/6

OBJETIVOS

Desarrollar competencias básicas para:

1. Aprovechar los mecanismos de la herencia y de interfaces.
2. Organizar las fuentes en paquetes.
3. Usar la utilidad `jar` de java para entregar una aplicación.
4. Extender una aplicación cumpliendo especificaciones de diseño, estándares y verificando su corrección.
5. Vivenciar las prácticas XP :  Code must be written to agreed [standards](#). Code the [unit test first](#).
6. Utilizar los programas básicos de java (javac, java, javadoc, jar), desde la consola.

ENTREGA

- Incluyan en un archivo **.zip** los archivos correspondientes al laboratorio. El nombre debe ser los dos apellidos de los miembros del equipo ordenados alfabéticamente.
- Deben publicar el avance al final de la sesión y la versión definitiva en la fecha indicada en los espacios preparados para tal fin.

DESARROLLO

Contexto

El modelo clásico de **Lotka-Volterra** fue propuesto originalmente para explicar variaciones en las poblaciones de peces en el Mediterráneo, pero desde entonces se ha utilizado para explicar la dinámica de cualquier sistema depredador-presa en el cual ciertas suposiciones sean válidas.

[\[VER SIMULADOR\]](#)

El propósito del paquete domino es encapsular la lógica central del simulador schelling donde contiene clases que representan los elementos principales del sistema, como la ciudad, las personas y los estados.

Nuestro sistema Lotka-Volterra será de lobos (predadores) , ovejas (presa) y heno (recursos). Todos habitan un valle, representado en una cuadrícula bidimensional.

El valle se modifica en una secuencia de pasos. En cada paso cada unidad (lobo, oveja o heno), que no ha actuado, atúa en orden de izquierda a derecha y de norte a sur.

Conociendo [En lab03.doc y valley.asta]

1. En el directorio descarguen los archivos contenidos en **valley.zip**. Revisen el código:
a) ¿Cuántos paquetes tiene?

El archivo valley.zip tiene dos paquetes, que son presentación y dominio.

b) ¿Cuál es el propósito del paquete presentación?

El propósito del paquete presentación es una aplicación que maneja la interfaz gráfica. En el simulador valley, contiene paquetes que contienen clases que se encargan de mostrar la simulación en pantalla y gestionar eventos.

c) ¿Cuál es el propósito del paquete dominio?

El propósito del paquete domino es encapsular la lógica central del simulador valley donde contiene clases que representan los elementos principales del ejemplo, animales, mamíferos, el valle, etc.

2. Revisen el paquete de dominio,

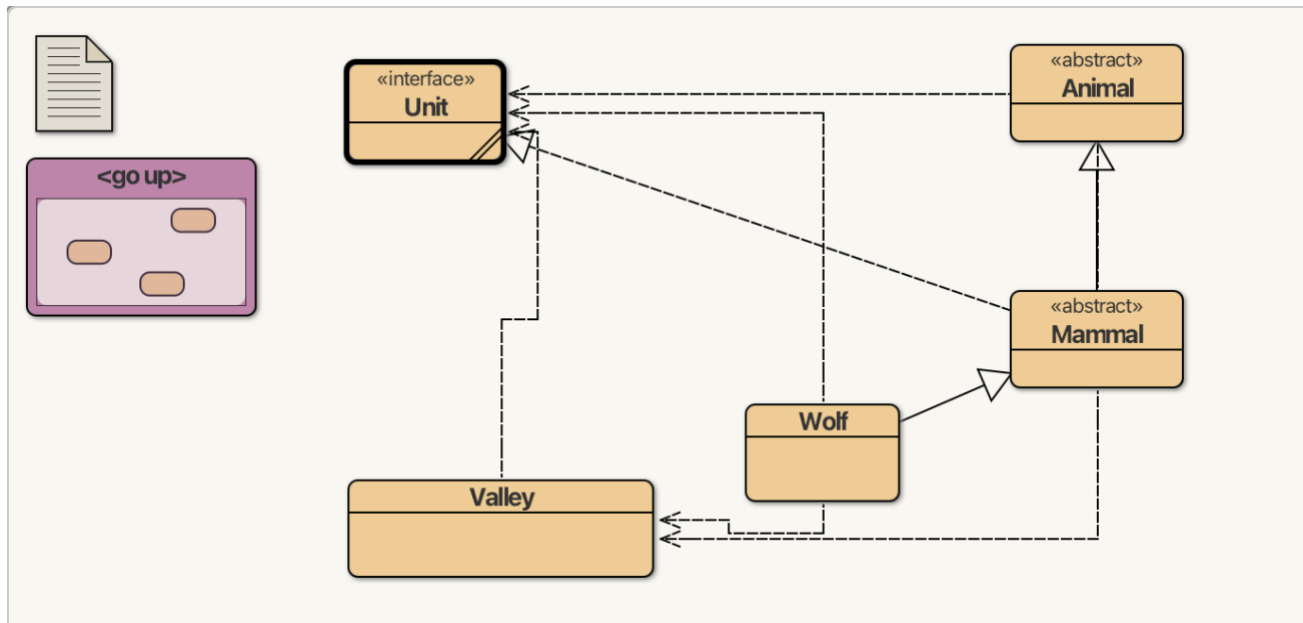
a) ¿Cuáles son los diferentes tipos de componentes de este paquete?

Tiene tres tipos de componentes, tenemos la clase estándar, tenemos la clase abstracta y, por último, la interfaz.

Interface: Unit

Abstrac: Animal y Mammal

Normales: Valley y Wolf



b) ¿Qué implica cada uno de estos tipos de componentes?

Interface (Unit): Define un contrato que otras clases deben seguir. No tiene implementación, solo métodos abstractos que las clases que la implementen deben definir.

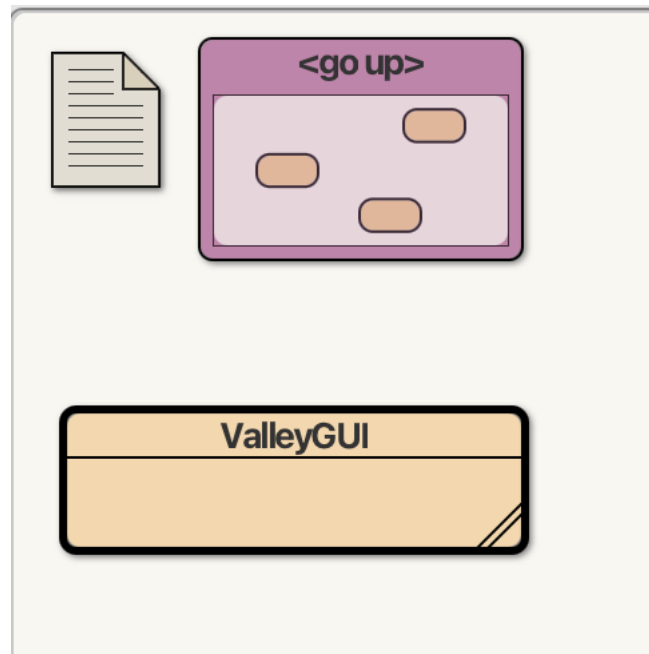
Clase abstracta (Animal, Mammal): Es una clase que puede contener tanto métodos abstractos (sin implementación) como métodos concretos. Sirve como base para otras clases, pero no se puede instanciar directamente.

Clases normales (Valley, Wolf): Son clases concretas que pueden ser instanciadas y contienen la lógica específica del sistema. Pueden heredar de una clase abstracta o implementar interfaces.

3. Revisen el paquete de presentación,

a) ¿Cuántos componentes tiene?

En este caso tiene un solo componente que es una clase llamada “ValleyGUI”. Sin embargo, al abrir el código podemos determinar que tiene otra clase llamada “PhotoValley”. Que es el que extiende JPanel y maneja la representación gráfica de la ciudad



```
public class ValleyGUI extends JFrame{
```

```
class PhotoValley extends JPanel{
```

Solo tiene un componente, la clase estándar.

b) ¿Cuántos métodos públicos propios (no heredados) ofrece?

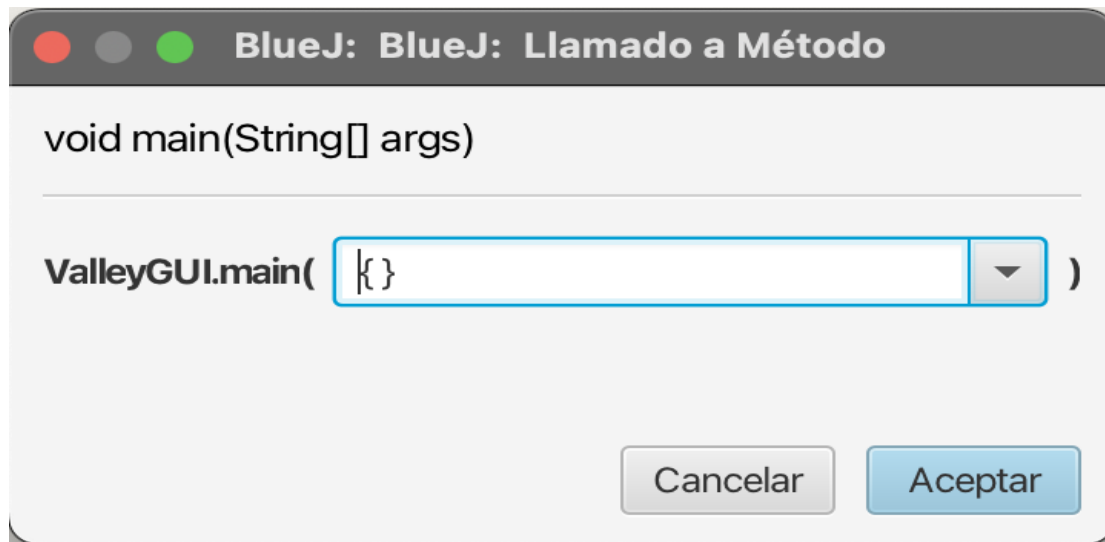
Estos son los únicos métodos públicos definidos en ValleyGUI, por lo que tiene 2 métodos públicos propios y no heredados.

```
public Valley gettheValley(){
    return theValley;
}
```

```
public static void main(String[] args) {
    ValleyGUI cg=new ValleyGUI();
    cg.setVisible(true);
}
```

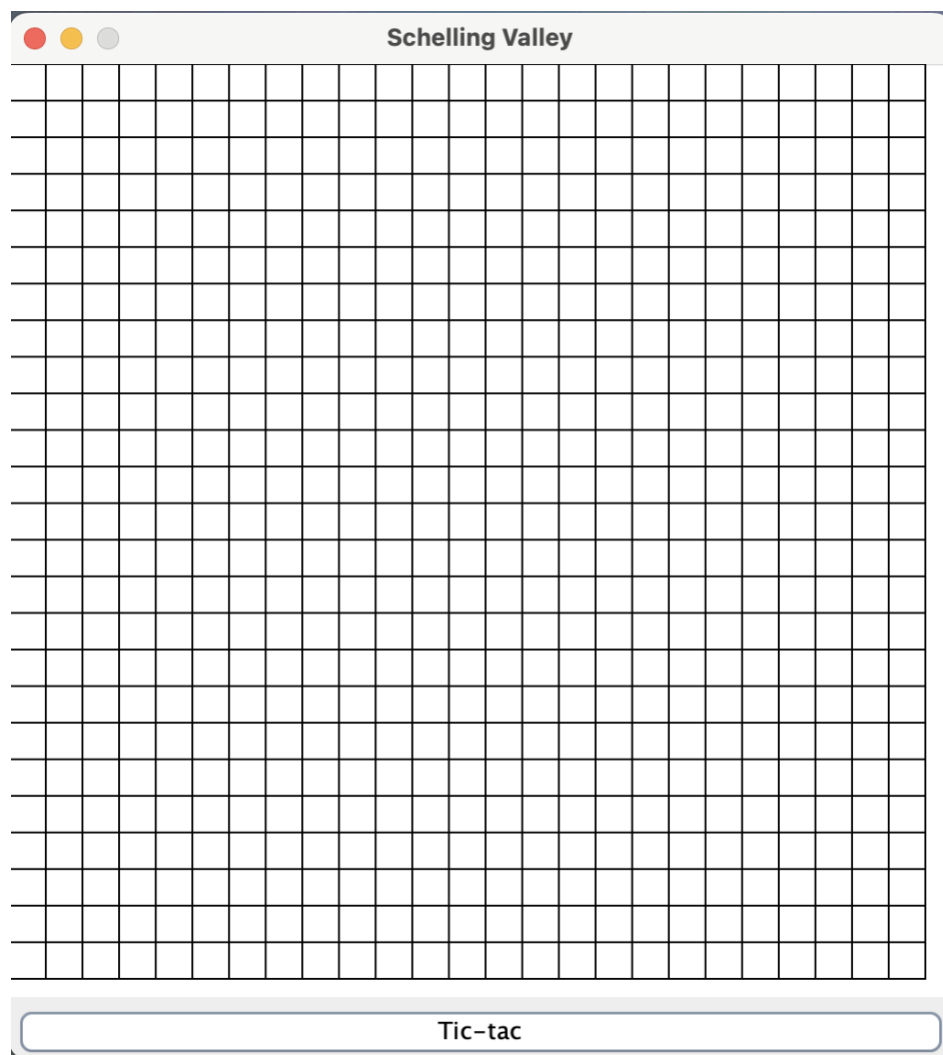
4. Para ejecutar un programa en java, ¿Qué método se debe ejecutar? ¿En qué clase se encuentra?

Para ejecutar el programa se debe ejecutar el método void main(String[] args), se encuentra en el paquete presentation, en la clase ValleyGUI.



5. Ejecuten el programa. ¿Qué funcionalidades ofrece? ¿Qué hace actualmente? ¿Por qué?
(Deben ejecutar la aplicación java, no crear un objeto como lo veníamos haciendo)

Cuando se ejecuta el método main, encontramos un tablero donde se ejecuta el Simulador:



El botón Tic-tac no realiza nada.

Arquitectura general. [En lab03.doc y valley.asta]

1. Consulte el significado de las palabras `package` e `import` de java. ¿Qué es un paquete?
¿Para qué sirve? ¿Para qué se importa? Explique su uso en este programa.

En Java:

- **Package (paquete):** Es una forma de organizar clases y evitar conflictos de nombres. Funciona como una carpeta que agrupa clases relacionadas. Se declara con `package nombre_del_paquete;` al inicio de un archivo Java.
- **Import (importar):** Sirve para traer clases o paquetes de otras partes del código o bibliotecas externas, evitando escribir rutas completas. Se usa con un `import nombre_del_paquete.NombreDeClase;` o `import nombre_del_paquete.*;` para importar todas las clases de un paquete.

¿Para qué sirve un paquete?

Un paquete en Java agrupa clases relacionadas para organizar mejor el código, evitar conflictos de nombres y mejorar la reutilización. Permite estructurar proyectos grandes dividiéndolos en módulos más manejables.

¿Para qué se importa?

Se importa un paquete en Java para poder usar sus clases y métodos sin escribir su ruta completa. Esto hace el código más claro, facilita la reutilización de código y permite acceder a bibliotecas estándar o externas de manera más sencilla.

Uso en el programa:

Package:

Se evidencia que cada archivo comienza con la declaración:

```
package domain;
```

Esto significa que todas las clases (Agent, Item, City, Person) pertenecen al paquete domain.

Lo que permite estructurar el código en paquetes lógicos, evitar conflictos con respecto a los nombres con otras clases y paquetes. También permite definir la visibilidad de clases, métodos y atributos entre diferentes paquetes.

Import:

En este caso cuando queremos usar clases de otros paquetes, entonces es necesario el `import`. Se utiliza de dos maneras en el código.

1. Importar de otra clase:

```
import java.awt.Color;  
import java.util.*;
```

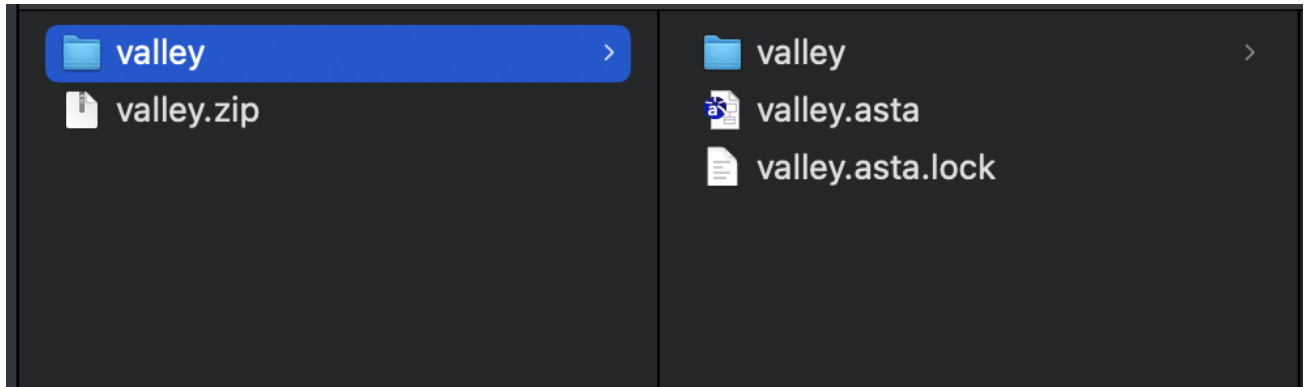
2. Importar dentro del mismo paquete:

```
import domain.City;  
import domain.Person;
```

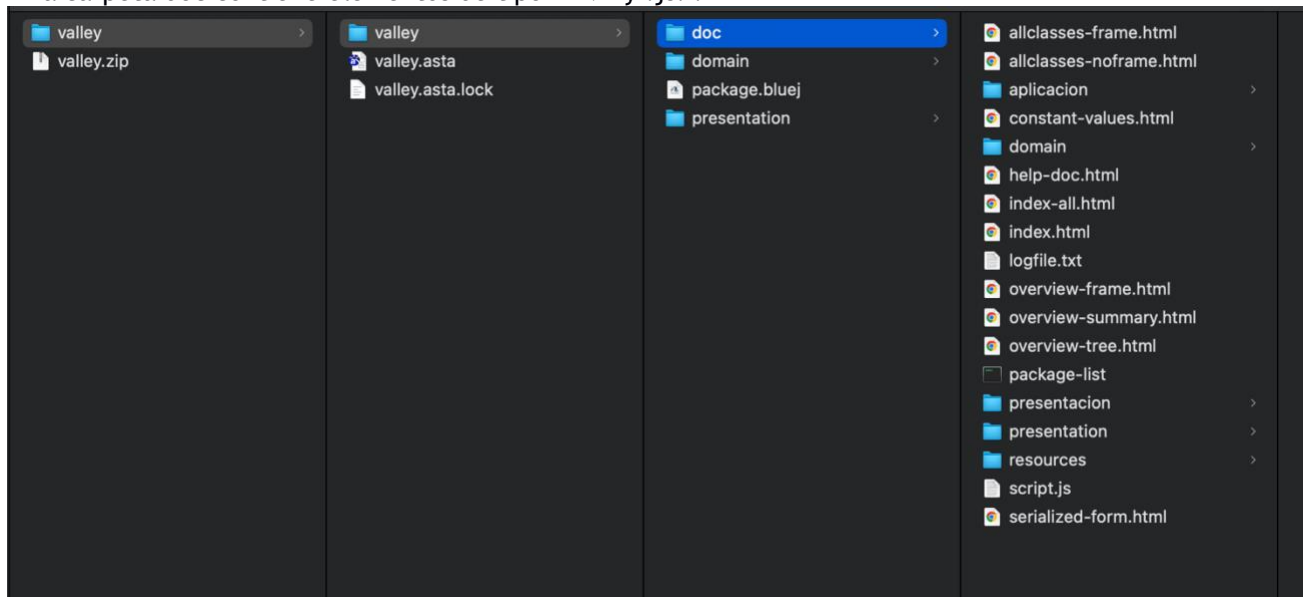
2. Revise el contenido del directorio de trabajo y sus subdirectorios. Describa su contenido.

¿Qué coincidencia hay entre paquetes y directorios?

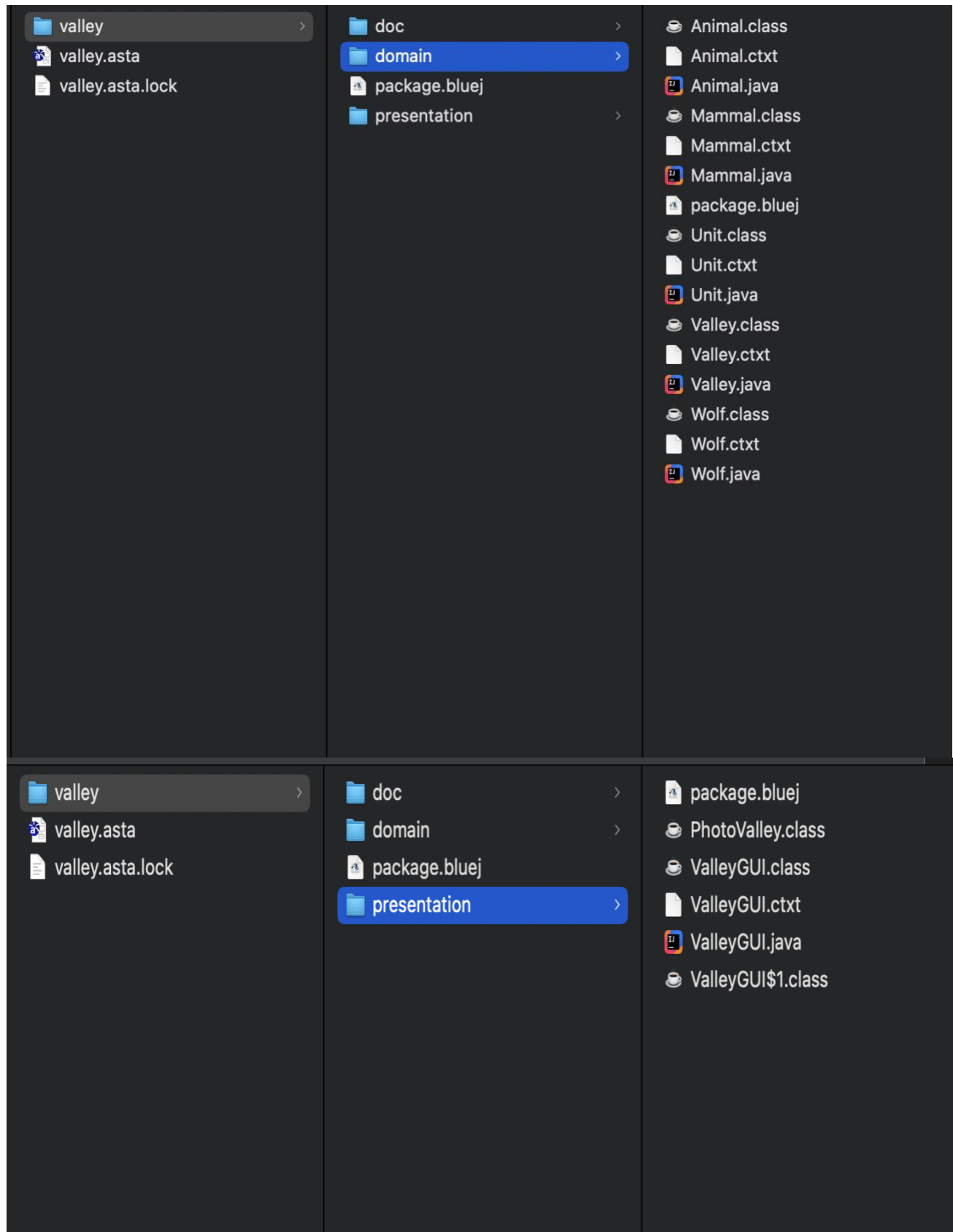
- Dentro del directorio encontramos lo siguiente:



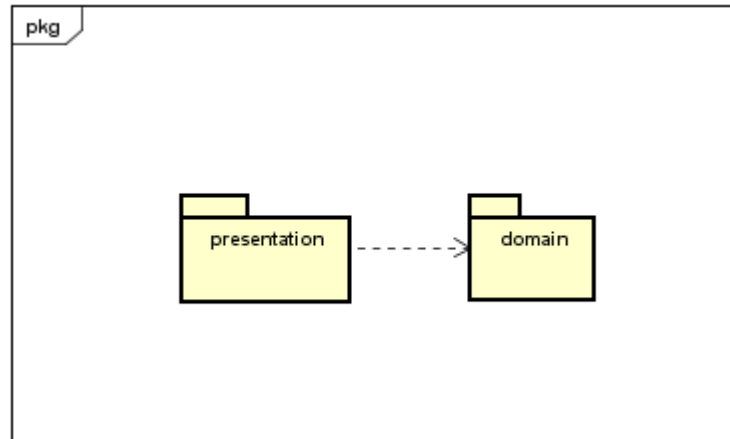
- La carpeta doc contiene elementos de tipo HTML y .js. :



- En la carpeta Domain y Presentation encontramos las clases del paquete dominio y presentation, ambas carpetas cuentan con los métodos y archivos.ctxt encargados de la documentación del código.



3. Adicione al diseño la arquitectura general con un diagrama de paquetes en el que se presente los paquetes y las relaciones entre ellos. Consulte la referencia en moodle.
En astah, crear un diagrama de clases (cambiar el nombre por Package Diagram0)

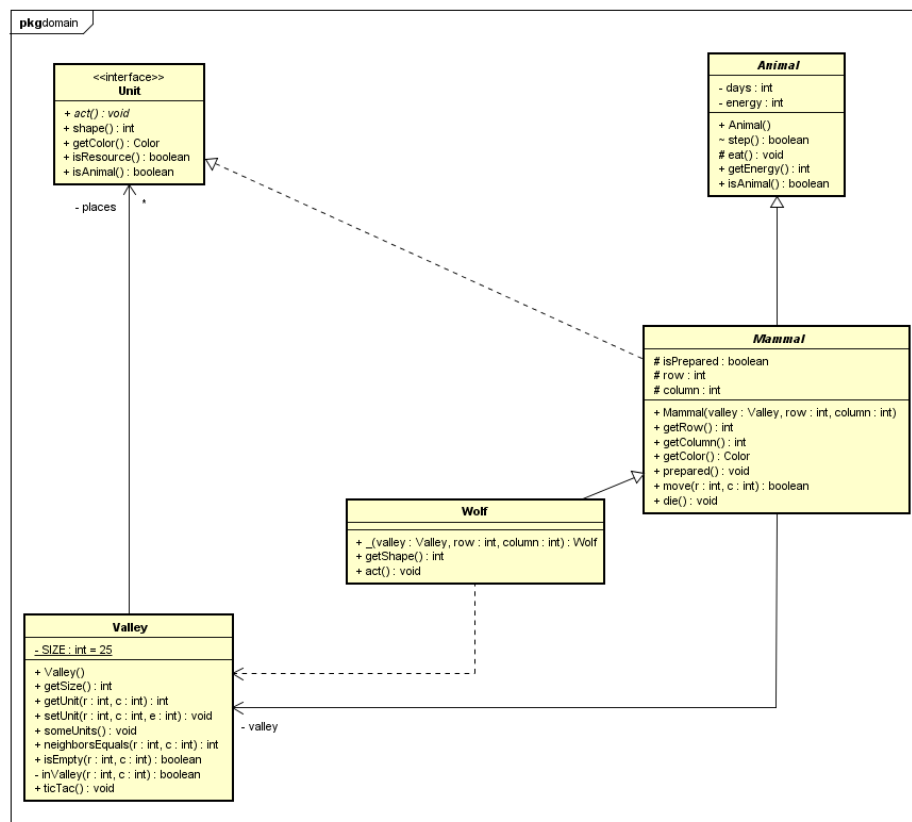


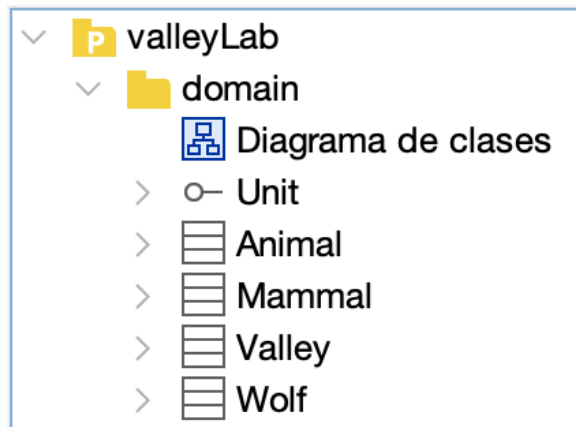
Arquitectura detallada. [En lab03.doc y valley.asta]

1. Para preparar el proyecto para **BDD**. Completen el diseño detallado del paquete de dominio. Adicionen el diagrama de clases en el paquete correspondiente.

a) ¿Qué componentes hacían falta?

Hacian falta la clase Wolf y la interfaz Unit.

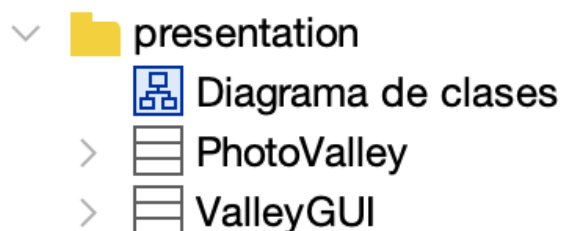
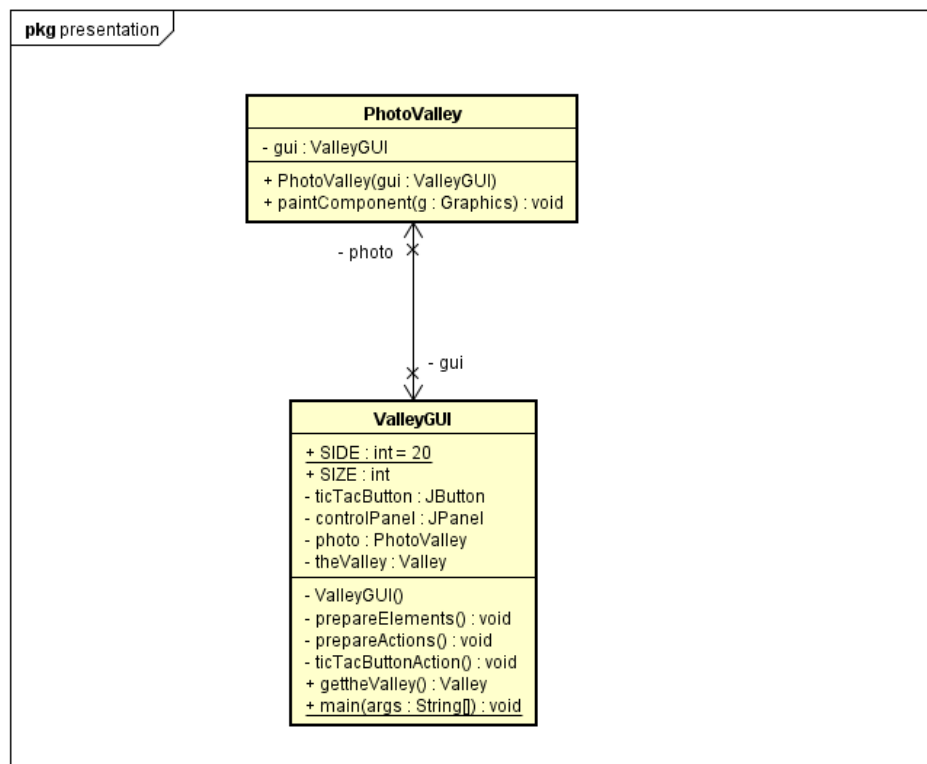




2. Completen el diseño detallado del paquete de presentación. Adicionen el diagrama de clases al paquete correspondiente.

a) ¿Por qué hay dos clases y un archivo Java?

Porque PhotoValley es una clase interna de ValleyGUI, PhotoValley no cumple una función necesaria como para necesitar ser una clase aparte en la aplicación. Si una clase solo tiene sentido dentro del contexto de otra, usar una clase interna ayuda a mantener esa relación clara.



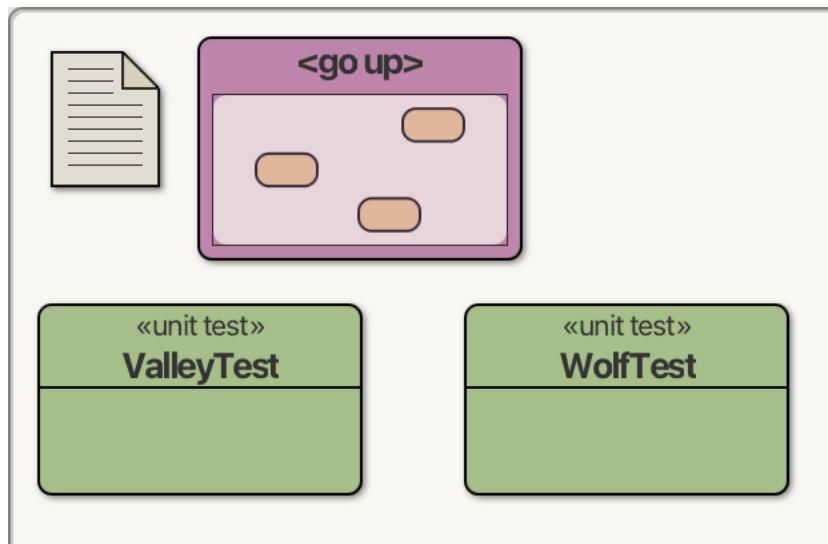
3. **Adicione la clase de pruebas unitarias necesaria para BDD en un paquete independiente de test. (No lo adicione al diagrama de clases)**

a. **¿Qué clase debe importar?**

Se debe importar en las clases del paquete domain, ya que en este paquete está toda la lógica de la aplicación, sobre todo en Valley.

b. **¿Por qué?**

Porque es la clase principal, ya que implementa las demás clases para el funcionamiento es decir es la fachada de la aplicación.



✓ Test.WolfTest.testWolfInitialEnergy()
✓ Test.WolfTest.testWolfActDecreaseEnergy()
✓ Test.WolfTest.testWolfCreation()
✓ Test.WolfTest.testWolfShape()
✓ Test.WolfTest.testWolfsAnimal()
✓ Test.WolfTest.testWolfPositionInValley()
✓ Test.WolfTest.testWolfInValley()

Ejecuciones: 7

✗ Errores: 0

✗ Fallos: 0

Tiempo Total: 2ms

Mostrar Fuente

Cerrar

✓ Test.ValleyTest.testTicTacExecutesAllUnits()
✓ Test.ValleyTest.testNeighborsEqualsWithoutSameClass()
✓ Test.ValleyTest.testMultipleUnitsRetrieval()
✓ Test.ValleyTest.testNeighborsEqualsMaximum()
✓ Test.ValleyTest.testValleyHasEmptySpaces()
✓ Test.ValleyTest.testIsEmptyWithEmptyPosition()
✓ Test.ValleyTest.testGetUnit()
✓ Test.ValleyTest.testNeighborsEqualsOutOfBounds()
✓ Test.ValleyTest.testNeighborsEqualsWithSameClass()
✓ Test.ValleyTest.testNeighborsEqualsOnEmptyPosition()
✓ Test.ValleyTest.testIsEmptyOutOfBounds()
✓ Test.ValleyTest.testIsEmptyWithOccupiedPosition()
✓ Test.ValleyTest.testValleySize()

Ejecuciones: 13

✗ Errores: 0

✗ Fallos: 0

Tiempo Total: 1ms

Mostrar Fuente

Cerrar

Ciclo 1. Iniciando con los lobos [En lab03.doc y *.java]

(NO OLVIDE BDD - MDD)

1. Estudie la clase `Valley`
¿Qué tipo de colección usa para albergar cosas?

La clase Valley usa una matriz 2D (`Unit[][] places`) de tamaño fijo (`SIZE`) como estructura de almacenamiento.

¿Puede recibir lobos?

Sí. Los Wolf (que extienden `Mammal` e implementan `Unit`) se almacenan directamente en `places[row][col]` mediante `setUnit(r,c,u)` o indirectamente desde el constructor del `Mammal`, que llama `valley.setUnit(row,column,this)`.

¿Por qué?

Porque la matriz está tipada como `Unit` y `Wolf` implementa/actúa como `Unit`, por tanto es compatible. Además Valley expone métodos `getUnit`, `setUnit`, `inValleyPublic` que permiten colocar y consultar unidades en posiciones concretas.

2. Estudie el código asociado a la clase `Wolf`,

- a) ¿de qué color es?

En la implementación el Wolf establece `color = Color.black`; por tanto negro.

```
public Wolf(Valley valley,int row, int column){
    super(valley,row, column);
    color=Color.black;
}
```

- b) ¿qué forma usa para pintarse?

Hereda `shape()/getShape()` de `Mammal/Unit` y por defecto retornamos `Unit.ROUND` (forma circular).

```
public class Wolf extends Mammal{
    public Wolf(Valley valley,int row, int column){
        super(valley,row, column);
        color=Color.black;
    }

    public int getShape(){
        return Unit.ROUND;
    }

    public void act(){
        if (getEnergy()==0){
            die();
        }else{
            if (! move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1)){
                move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1);
            }
        }
    }
}
```

c) ¿cuándo pierde su energía?

La energía disminuye en cada step () (cada vez que act() llama a step ()). En la implementación step () resta 1 unidad de energía por tic (y si la energía llega a 0, el animal muere y se retira del Valley). Además, comer (método eat()) la incrementa hasta un máximo.

```
/**The animal makes one step
 *
 */
final boolean step(){
    boolean ok=false;
    if (energy>=1){
        energy-=1;
        ok=true;
    }
    return ok;
}

public void act(){
    if (getEnergy()==0){
        die();
    }else{
        if (! move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1)){
            move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1);
        }
    }
}
```

d) ¿qué componentes definen la clase **Wolf** ?

Wolf:

Atributos (posición row, column heredados de Mammal, color)

```
public class Wolf extends Mammal{

    public Wolf(Valley valley,int row, int column){
        super(valley,row, column);
        color=Color.black;
    }
}
```

Comportamiento (act()) — que puede ser el act() heredado o sobrescrito).

```
public void act(){
    if (getEnergy()==0){
        die();
    }else{
        if (! move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1)){
            move(row+ (int)(Math.random() * 3) - 1,column+ (int)(Math.random() * 3) - 1);
        }
    }
}
```

Constructor Wolf(Valley,row,column) que registra la unidad en el Valley.

```
public Wolf(Valley valley,int row, int column){
    super(valley,row, column);
    color=Color.black;
}
```

Wolf es una subclase de Mammal y por tanto hereda posicionamiento y la lógica básica; Wolf solo define color y, si se desea, comportamiento particular.

3. **Wolf** por ser un **Mammal**,

¿qué atributos tiene?

Atributos (heredados por Mammal): Referencia al Valley que lo contiene, row, column, isPrepared (bandera), color.

```
public abstract class Mammal extends Animal implements Unit{
    private Valley valley;

    protected int row,column;
    protected Color color;

    /**Create a new Mammal(<b>row,column</b>) in the valley <b>valley</b>..
     * @param valley
     * @param row
     * @param column
     */
    public Mammal(Valley valley,int row, int column){
        this.valley=valley;
        this.row=row;
        this.column=column;
        this.valley.setUnit(row,column,(Unit)this);
    }
}
```

¿qué puede hacer (métodos)?

act() (comportamiento por tic), move(r,c) (intentar moverse), getRow(), getColumn(), getColor(), prepared(), die() (morir y retirarse del valle). También hereda step(), eat(), getEnergy() desde Animal.

```
/**Returns the row
@return
*/
public final int getRow(){
    return row;
}
```

```
/**Returns the column
@return
*/
public final int getColumn(){
    return column;
}
```

```
/**Returns the color
@return
*/
public final Color getColor(){
    return color;
}
```

```
/**Move
*/
public boolean move(int r, int c){
    boolean ok=false;
    if (valley.isEmpty(r,c)){
        valley.setUnit(row, column,null);
        row=r;
        column=c;
        valley.setUnit(row,column,this);
        step();
        ok=true;
    }
    return ok;
}
```

```
/**Die
*/
public void die(){
    valley.setUnit(row, column,null);
}
```


¿qué decide hacer distinto (sobreescritura)?

Wolf puede sobreescribir `act()` para tener comportamiento específico (por ejemplo perseguir, cazar). En la implementación `act()` heredado o con llamada a `super.act()`.

¿qué no puede hacer distinto (métodos finales)?

En el diseño `getEnergy()` e `isAnimal()` son final (definidos en `Animal`) porque su semántica es universal y no deben romperse por subclases.

¿qué debe aprender a hacer (métodos abstractos)?

Si `Animal/Mammal` tuviera métodos abstractos (por ejemplo `act()` como abstracto), `Wolf` debería implementarlos. En el diseño `act()` no es abstracto (tiene un comportamiento por defecto).

4. Por comportarse como un `Unit`

¿qué sabe hacer?

`act()` (se ejecuta cada tic), `shape()` (forma para dibujar), `getColor()`, `isResource()` y `isAnimal()`. Es la abstracción que permite al `Valley` y `GUI` manipular cualquier entidad (recurso o animal).

¿qué decide hacer distinto?

Cada implementación (`Wolf`, `Mammal`, `Resource`, etc.) decide su propio `act()`, `getColor()`, `shape()`; `Wolf` implementa o sobreescribe ese comportamiento.

¿qué no puede hacer distinto?

La firma de los métodos establecidos por la interfaz no cambia; no puedes cambiar nombres o parámetros del contrato.

¿qué debe aprender a hacer?

Cualquier clase que implemente `Unit` debe implementar (o heredar implementaciones) de esos métodos para integrarse con `Valley` y la `GUI`.

5. De acuerdo con lo anterior un `Wolf`, ¿Cómo actúa?

1. Llama a `step()` → consume energía (-1). Si energía llega a 0 → `die()` y sale del valle.
2. Si sigue vivo, intenta moverse a una casilla aleatoria adyacente (incluye posibilidad de quedarse en la misma celda).
3. Si la casilla destino está vacía → se mueve.
4. Si la casilla destino contiene un recurso (un `Unit` con `isResource()==true` y no animal) → come (`eat()`, +20 energía) y ocupa la casilla.
5. Si hay otro animal u obstáculo → no se mueve. Es decir comportamiento simple: paso energético + movimiento aleatorio + comer si hay recurso. `Wolf` puede sobreescribir `act()` para comportamientos más complejos (caza, perseguir, reproducirse, etc.).

6. Ahora vamos a crear dos lobos en diferentes posiciones (10,10) (15,15) , llámelos **akela** y **larka** , usando el método `someUnits()` .

Ejecuten el programa:

¿Qué pasa con los lobos?

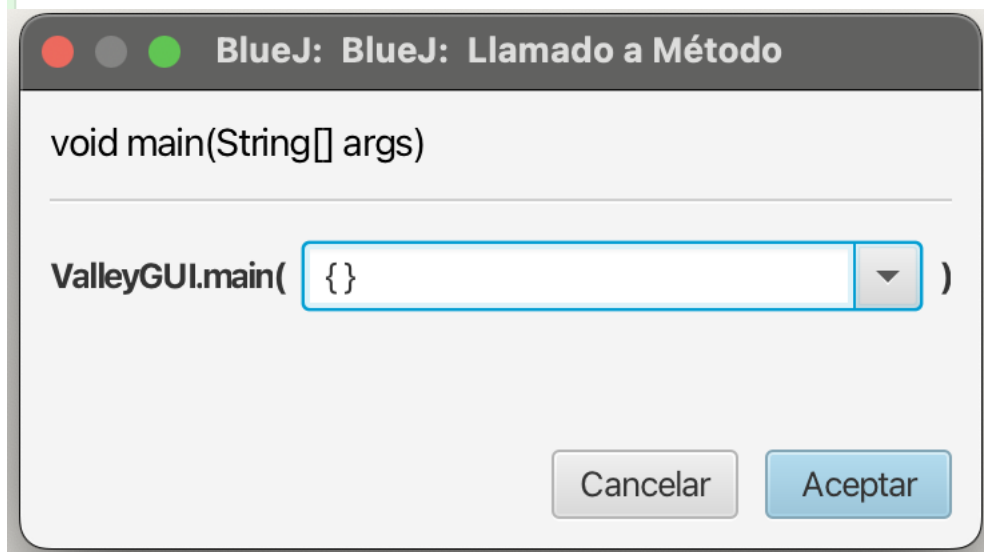
- En el estado inicial se ven dos W en las posiciones (10,10) y (15,15) cuando imprimimos el mapa simple.
- En cada `ticTac()` ambos lobos ejecutan `act()` (paso energético + intento de moverse).

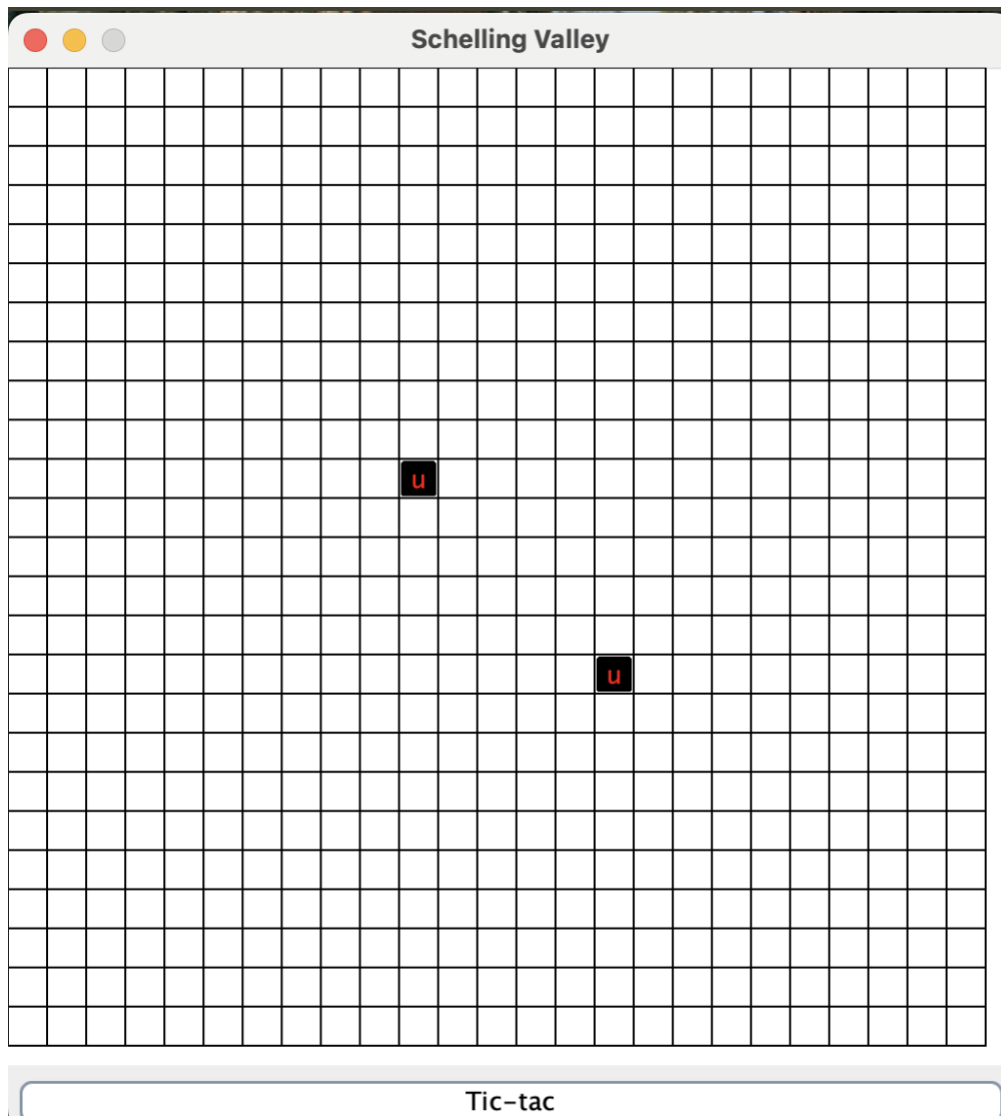
¿Por qué?

Porque `act()` reduce energía y mueve aleatoriamente; no hay fuente sostenida de comida en la versión mínima, por tanto terminan perdiendo energía eventualmente.

Capturen una pantalla significativa.

```
public void someUnits(){  
    new Wolf(this, 10, 10); // akela  
    new Wolf(this, 15, 15); // larka  
}
```





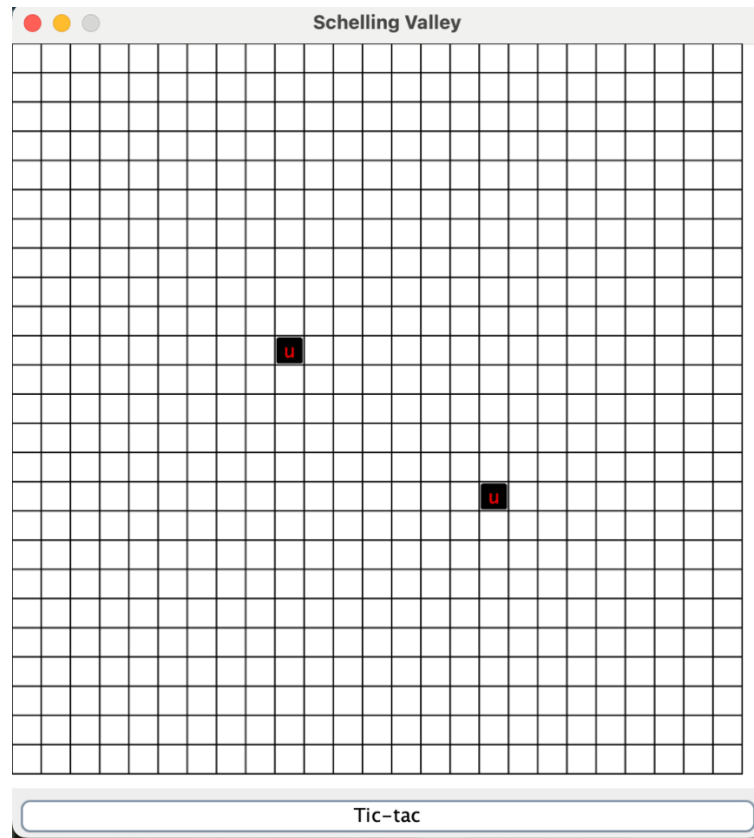
7. Diseñen, construyan y prueben el método llamado `ticTac()` de la clase `Valley`.

```
public void ticTac(){  
    for (int r = 0; r < SIZE; r++){  
        for (int c = 0; c < SIZE; c++){  
            if (places[r][c] != null){  
                places[r][c].act();  
            }  
        }  
    }  
}
```

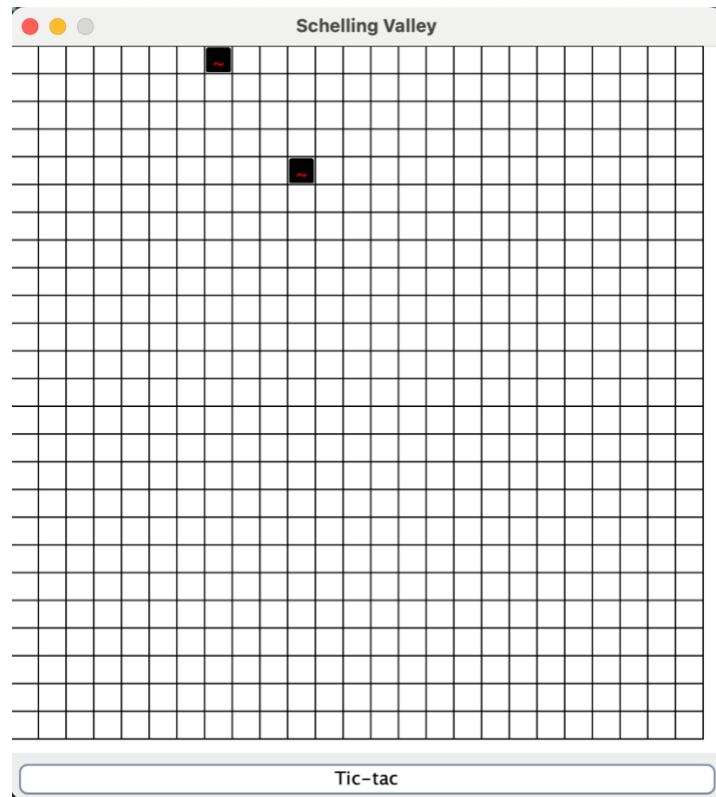
Luego realizamos una nueva prueba de unidad para comprobar que funcionara correctamente y visualizar el movimiento desde el método main.

8. ¿Cómo quedarían **akela y larka** después de uno, sesenta y ciento uno **Tic-tac**? Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes. ¿Es correcto?

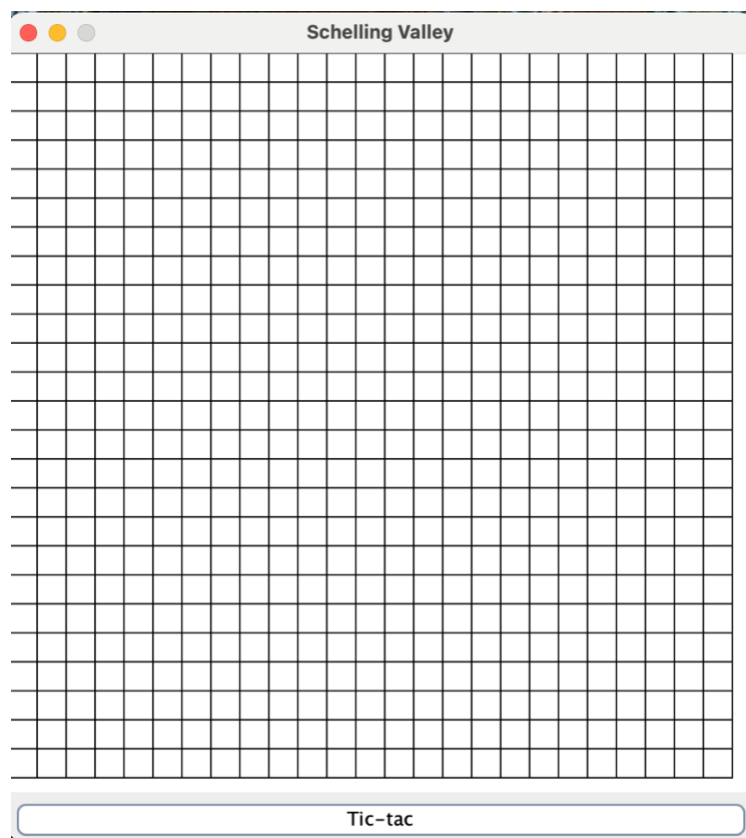
Cuando ocurre el uno (1) **Tic-tac** proceden a moverse a una posición aleatoria:



Cuando ocurre el sesenta (60) **Tic-tac** ocurre que les queda 40 puntos de energía:



Cuando ocurre el ciento uno (101) **Tic-tac** ocurre que se quedan sin energía y desaparecen:



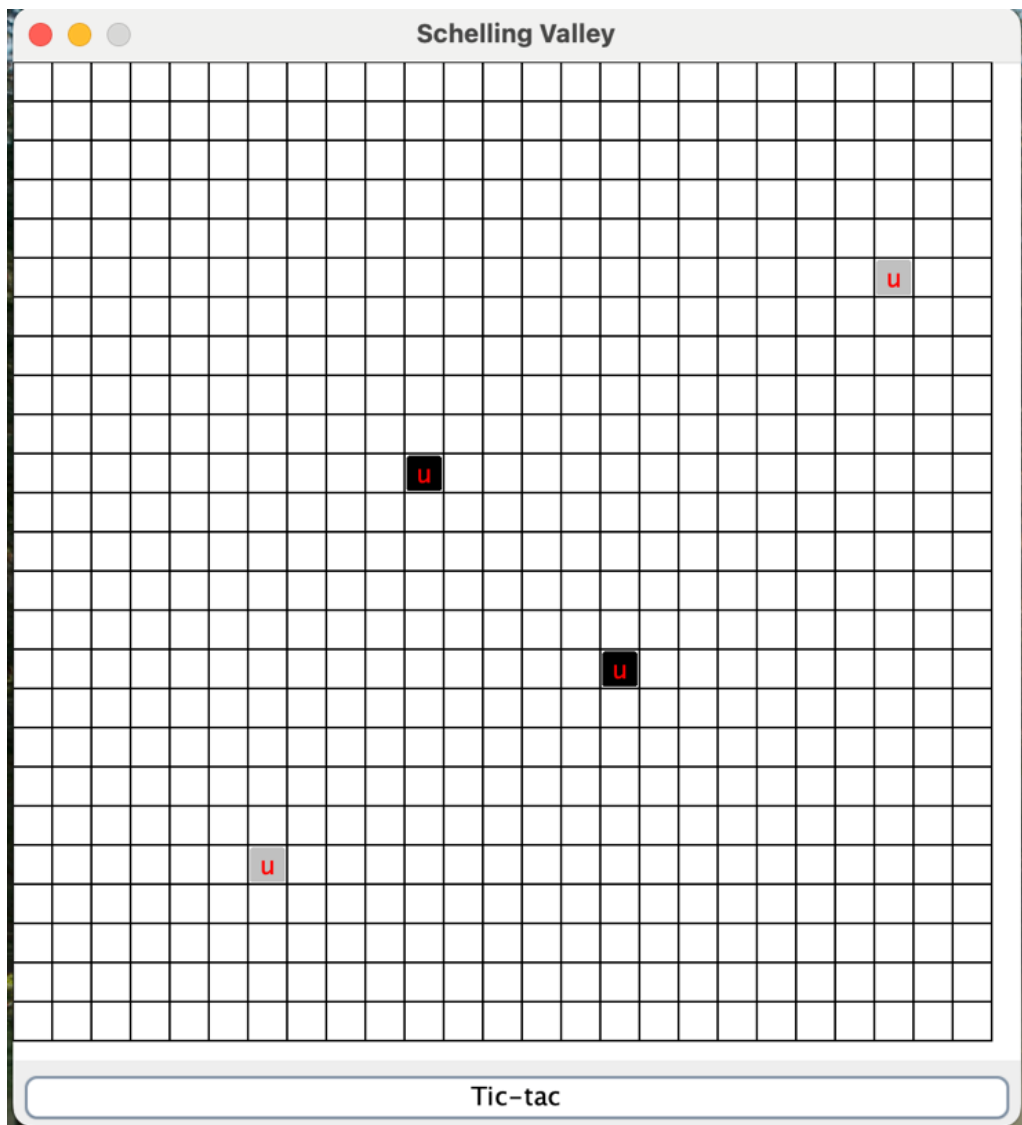
Ciclo 2. Incluyendo a las ovejas [En lab03.doc y valley.asta]

(NO OLVIDE BDD - MDD)

El objetivo de este punto es permitir recibir ovejas. Ellas (i) son rectángulos de color gris claro (ii) inician con 5 puntos de energía; (iii) se mueven en siempre hacia el norte y si se llegan al norte cambian de dirección camino hacia el sur (iv) cada paso pierden 1 punto de energía (v) al llegar al norte recuperan toda su energía (vi) si quedan vecinas a una oveja ganan 1 punto de energía (vii) si quedan vecinas a un lobo mueren.

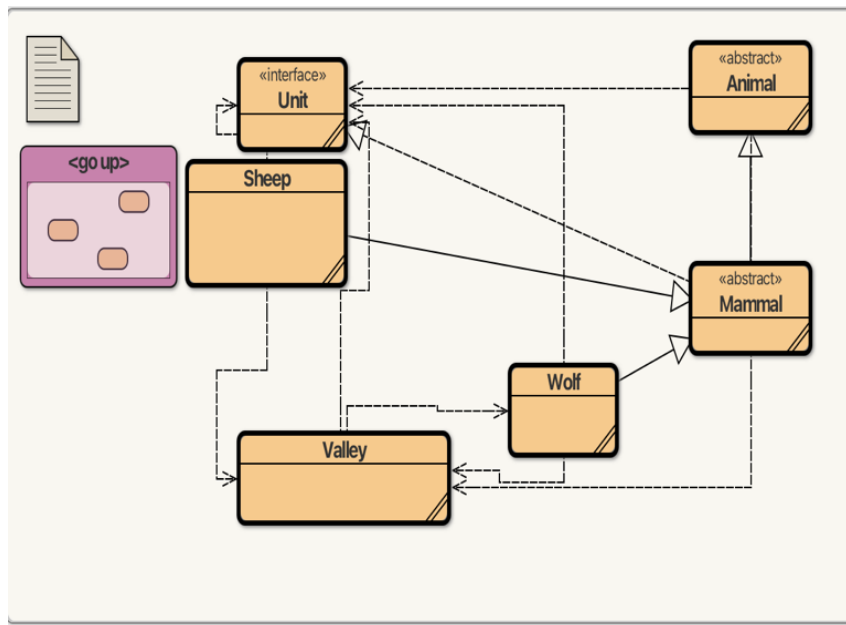
1. Para implementar este nuevo **Mammal** ¿cuáles métodos se sobre-escriben (overriding)?

Se sobrescriben los métodos `shape()` y `act()`, ya que el primero define la forma del objeto (en este caso, un rectángulo), mientras que el segundo establece un comportamiento distinto al tratarse de un nuevo tipo de animal.



2. Diseñen, construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)

Diseñamos la clase y desarrollamos nuevas pruebas unitarias para verificar su funcionamiento.



TestSheepTest.testSheepIsAnimal()

TestSheepTest.testSheepDiesWithZeroEnergy()

TestSheepTest.testSheepChangesDirectionAtNorth()

TestSheepTest.testSheepInValley()

TestSheepTest.testSheepPositionInValley()

TestSheepTest.testSheepMovesNorth()

TestSheepTest.testSheepLosesEnergyWhenMoving()

TestSheepTest.testSheepHasInitialEnergy()

TestSheepTest.testSheepCreation()

TestSheepTest.testSheepShape()

Ejecuciones: 10

Errores:0

Fallos:0

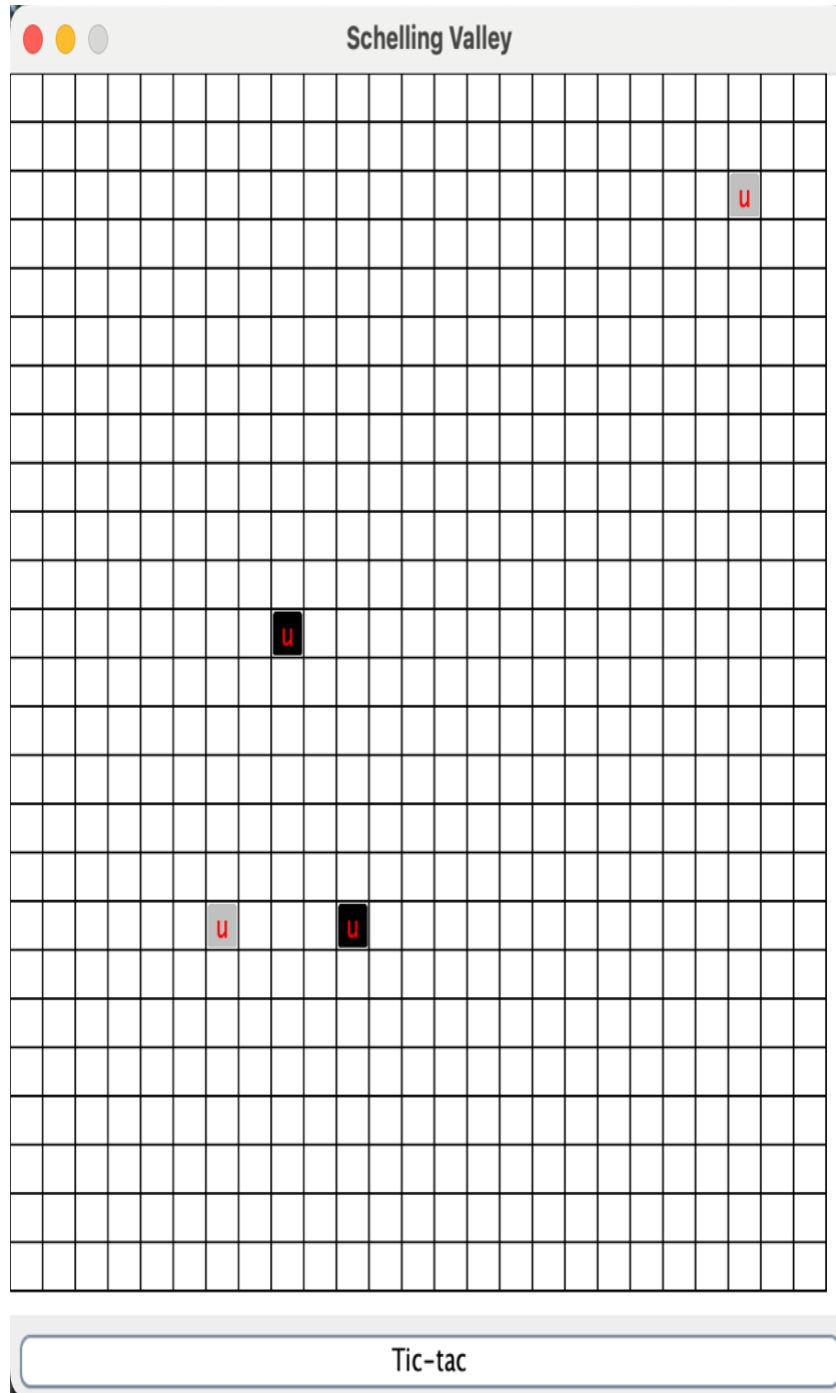
Tiempo Total: 2ms

Mostrar Fuente

Cerrar

3. Adicione una pareja de ovejas, llámelas **shaun** y **woolly**,
(a) ¿Cómo quedarían después de tres **Tic-tac**? Ejecuten el programa y hagan tres clics en el botón. Capturen una pantalla significativa.

Cuando ocurre el tercer (3) **Tic-tac** procede a quedar de la siguiente manera:



(b) ¿Es correcto?

Es correcto, ya que se desplazan hacia el norte y disminuyen su nivel de energía con cada movimiento.

Ciclo 3. Adicionando heno [En lab03.doc, valley.asta y *.java]

El objetivo de este punto es incluir paquetes de heno (sólo vamos a permitir el tipo básico de paquetes de heno) los paquetes de heno son cuadrados y van cambiando de color: rojo, amarillo, rojo, amarillo, rojo.

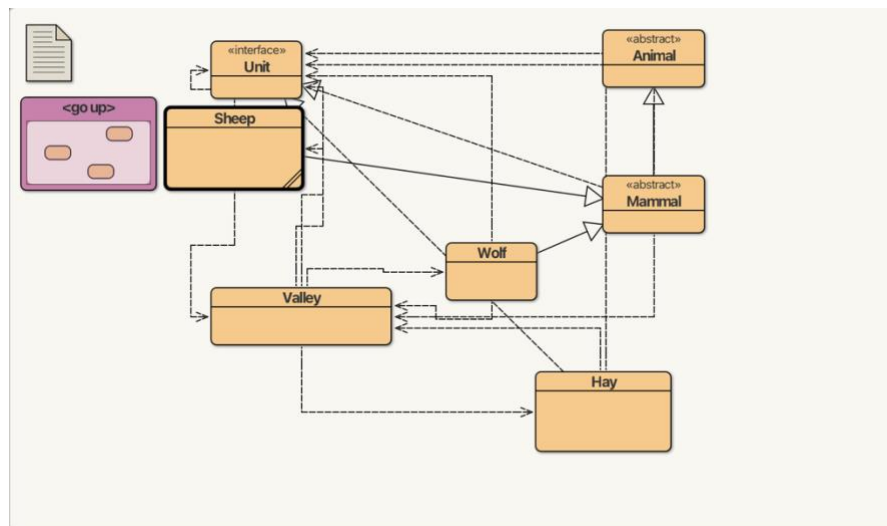
(NO OLVIDE BDD - MDD)

1. Para poder adicionar paquetes de heno, ¿debe cambiar en el código de `Valley` en algo? ¿por qué?

No es necesario realizar cambios, ya que Valley utiliza un arreglo `Unit[][]` que puede almacenar cualquier objeto que implemente la interfaz `Unit`. Dado que Hay implementará dicha interfaz, se integrará automáticamente en el sistema sin requerir modificaciones. Gracias al polimorfismo, Valley puede operar con cualquier tipo de objeto que sea una instancia de `Unit`.

2. Diseñen , construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)

Diseñamos la clase y desarrollamos nuevas pruebas unitarias para verificar su funcionamiento.



The screenshot shows a test runner interface with a list of unit tests that have passed. The tests are:

- ✓ TestHayTest.testHayCreationInitialColor()
- ✓ TestHayTest.testHayInValley()
- ✓ TestHayTest.testHayIsNotResource()
- ✓ TestHayTest.testHayPosition()
- ✓ TestHayTest.testHayShape()
- ✓ TestHayTest.testHayIsNotAnimal()
- ✓ TestHayTest.testHayColorCycle()

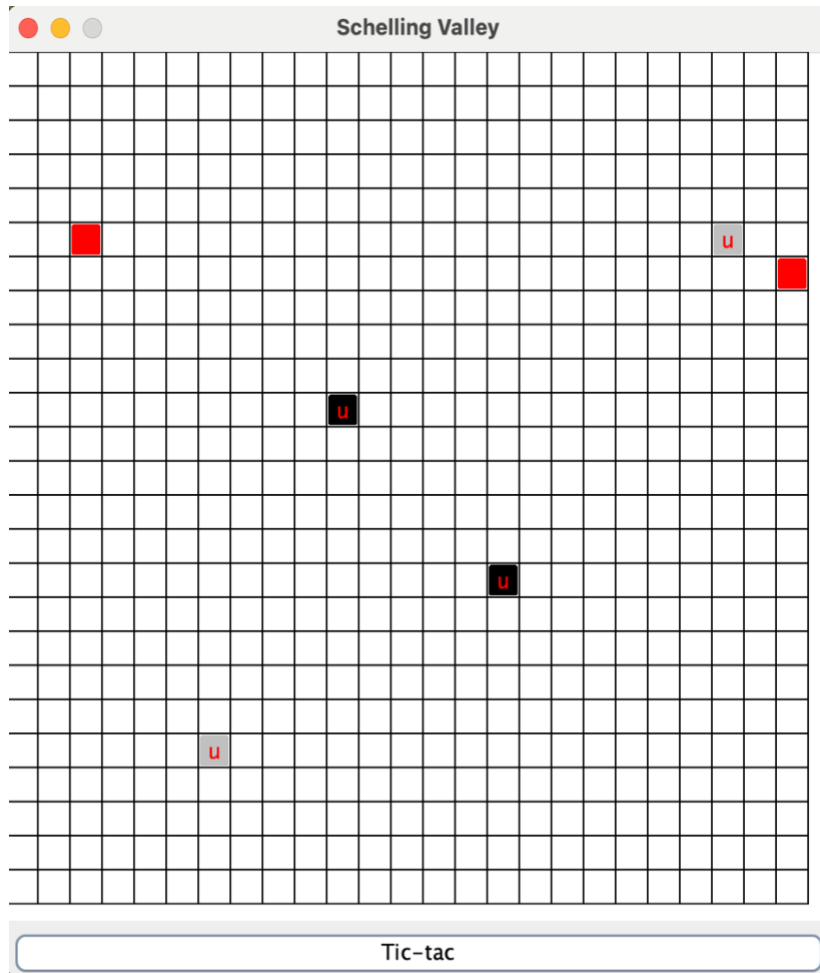
Below the list, a green bar indicates the overall test status. The summary bar shows:

- Ejecuciones: 7
- Errores: 0
- Fallos: 0
- Tiempo Total: 2ms

At the bottom, there are buttons for "Mostrar Fuente" and "Cerrar".

3. Adicionen dos paquetes de heno en las esquinas superiores de el valle, llámenlos `alarm` y `alert`,
- (a) ¿Cómo quedarían después de cuatro **Tic-tac**? Ejecuten el programa y hagan cuatro clics en el botón. Capturen una pantalla significativa.

Cuando ocurre el cuarto (4) **Tic-tac** procede a quedar de la siguiente manera:



(b) ¿Es correcto?

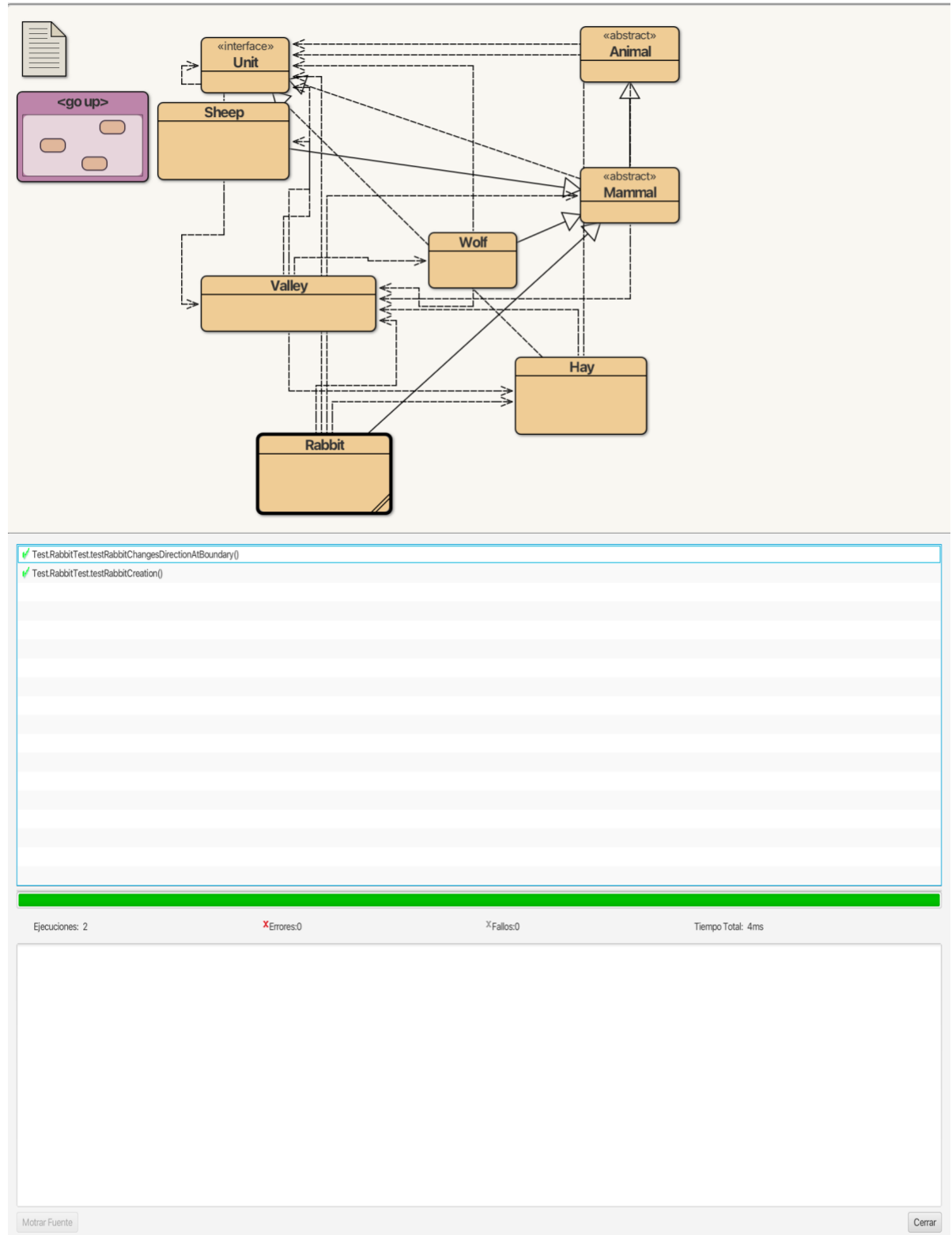
Sí. Los cuadrados rojos visibles en las esquinas superiores no se desplazan, pero cambian de color, lo cual corresponde al comportamiento esperado. Inician en color rojo y, tras cuatro ciclos de Tic-tac, alternan entre amarillo y rojo (rojo → amarillo → rojo → amarillo → rojo), finalizando nuevamente en rojo.

Ciclo 4. Nuevo Mamífero: Proponiendo y diseñando

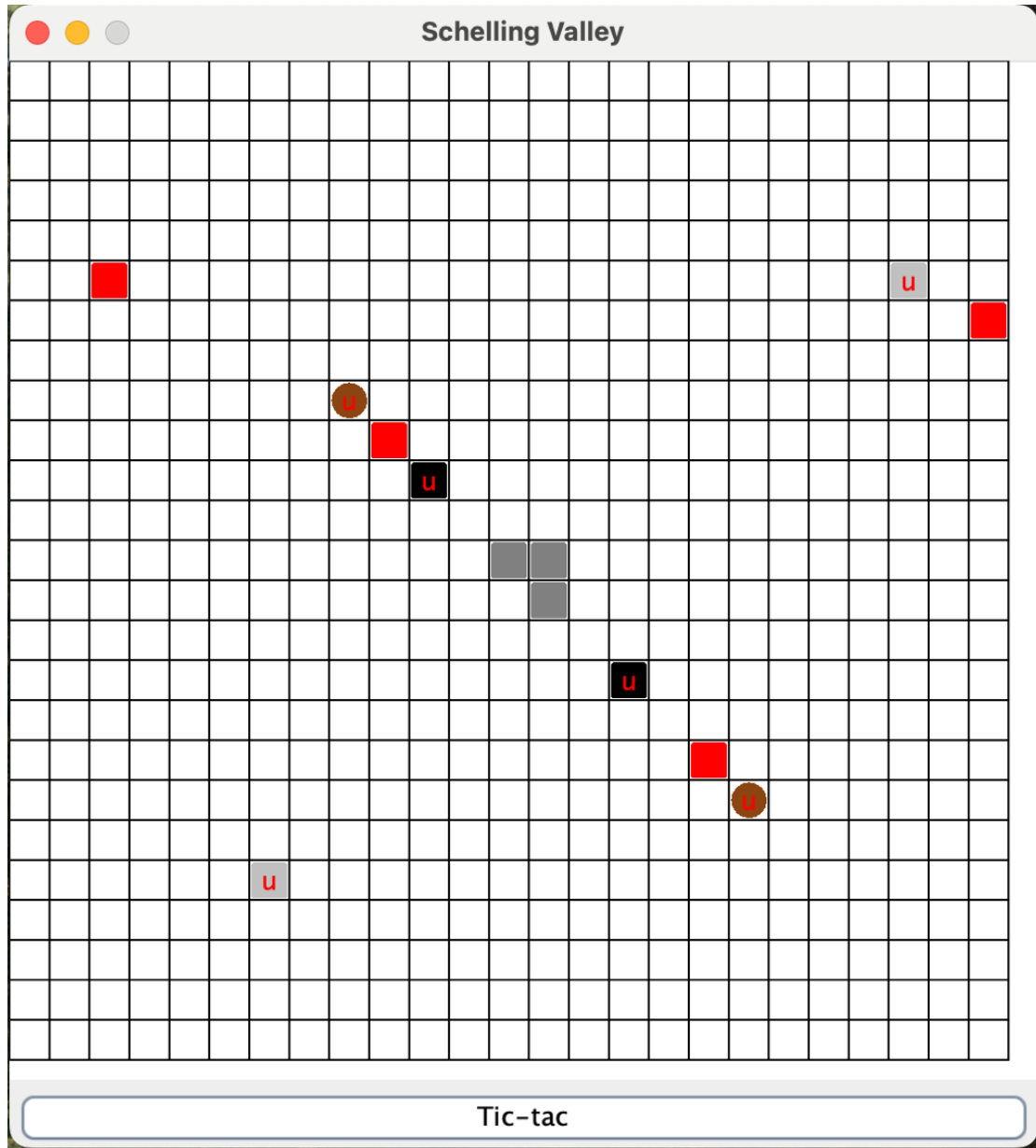
El objetivo de este punto es permitir recibir en un nuevo tipo de Mamífero.
(NO OLVIDE BDD - MDD)

1. Propongan, describan e implementen el nuevo tipo de Mamífero.
(Mínimo dos pruebas de unidad)

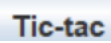
Se propuso un nuevo mamífero que es Conejo es decir la clase Rabbit:

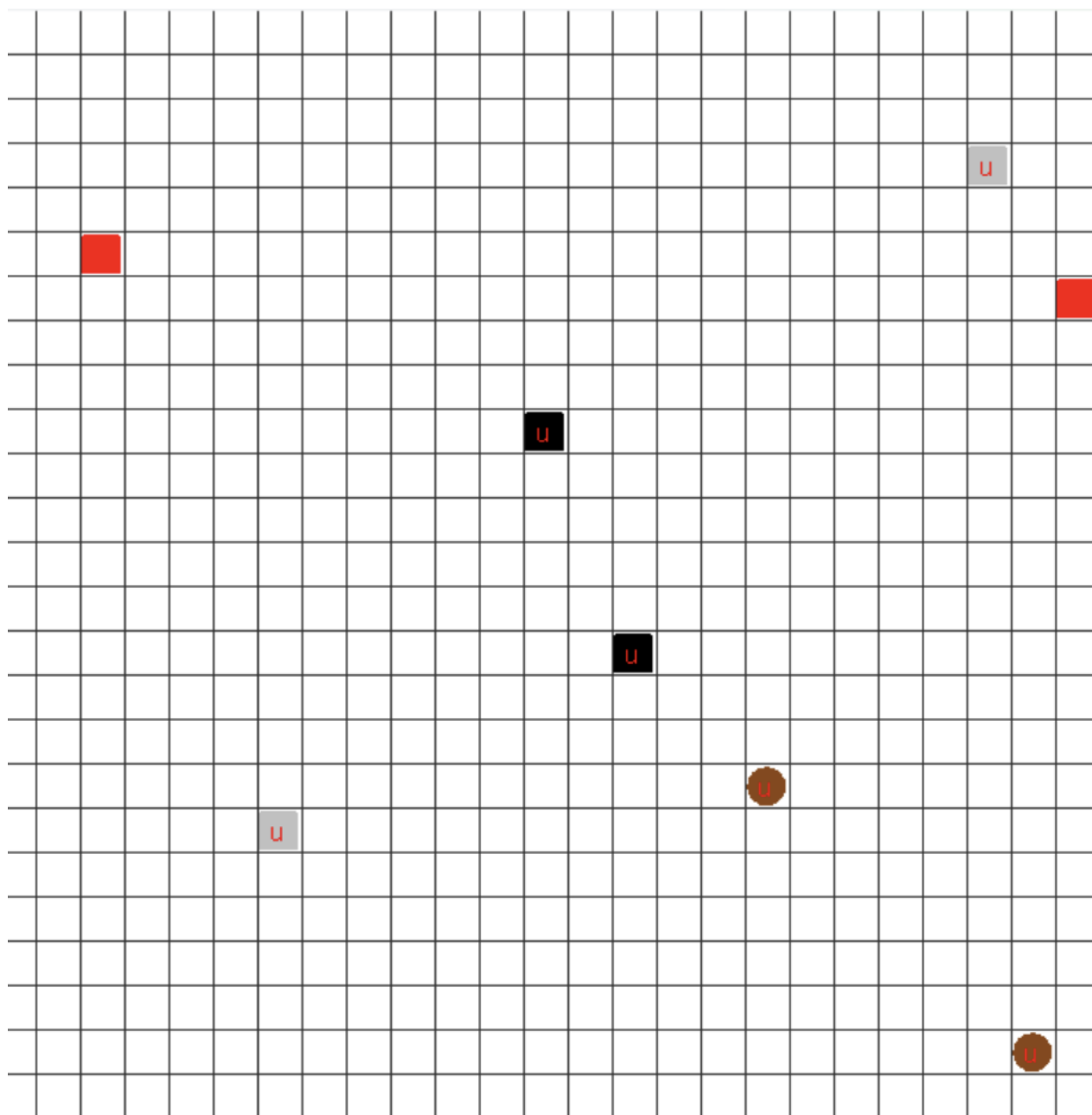


2. Considerando una pareja de ellos con el apellido de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba.



- (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.



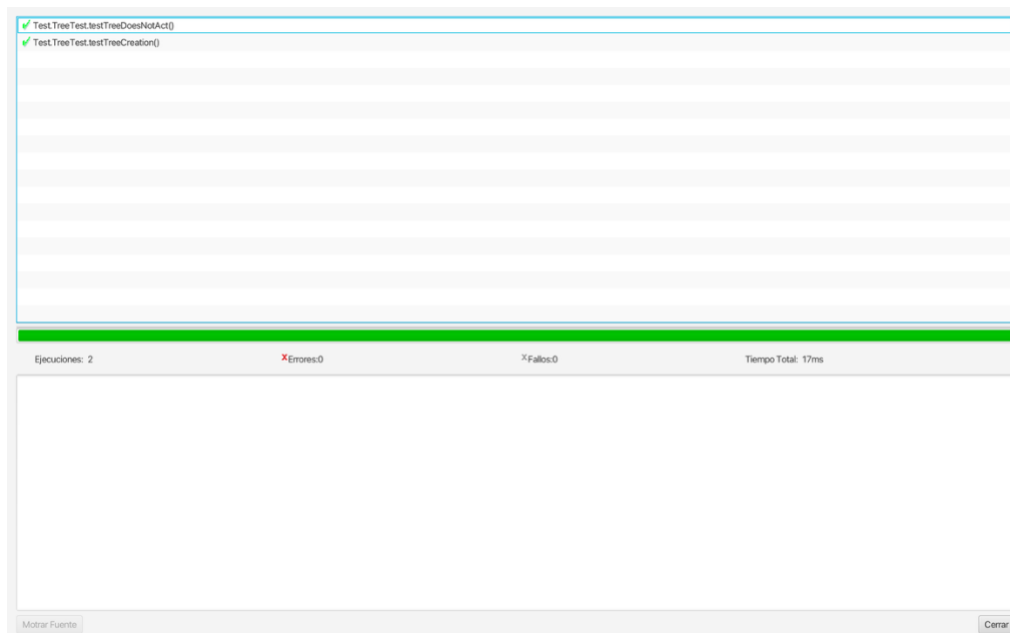
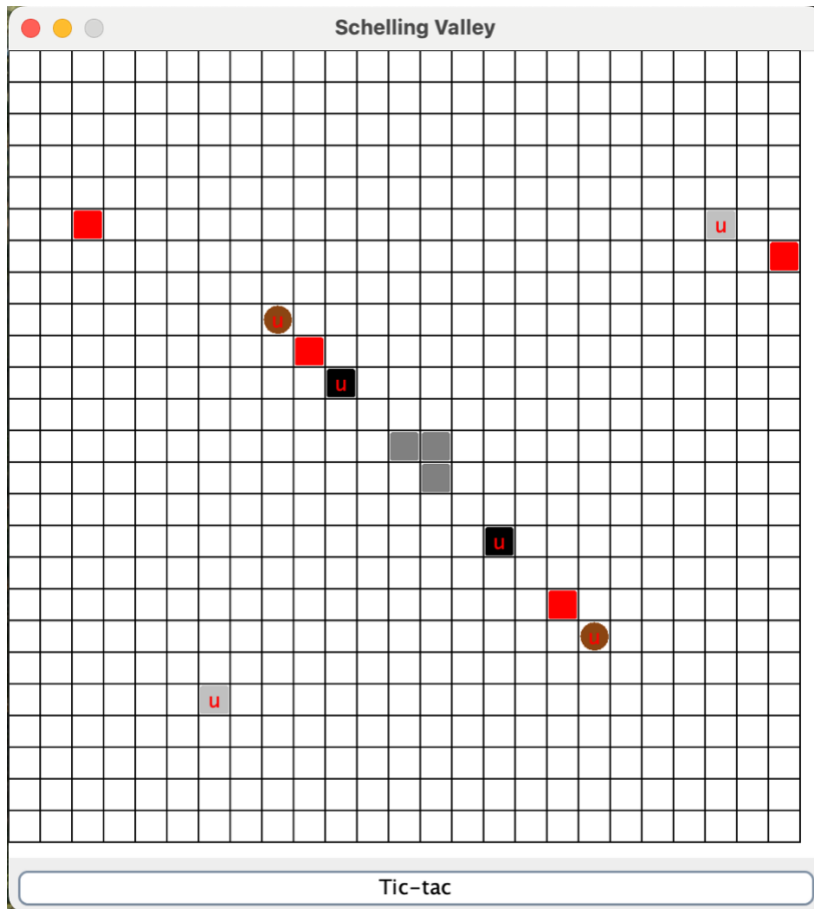


Tic-tac

Ciclo 5. Nuevo Unit: Proponiendo y diseñando

El objetivo de este punto es permitir recibir un nuevo Unit (no Animal) en el valle
(**NO OLVIDE BDD - MDD**)

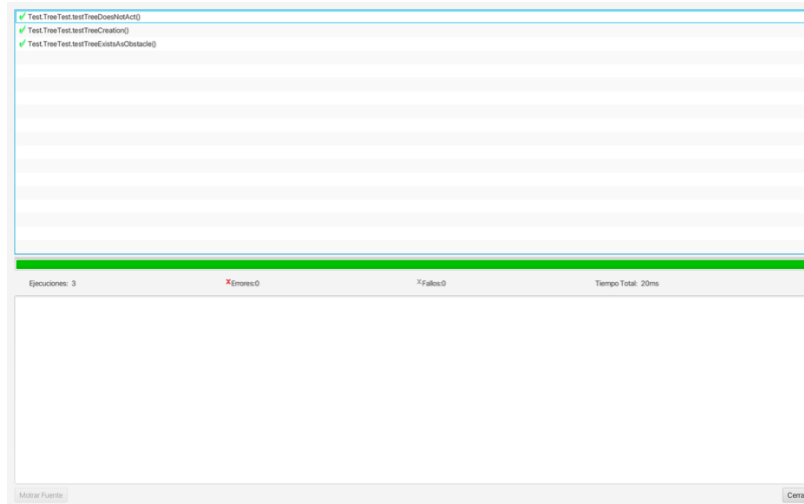
1. Propongan, describan e implementen el nuevo tipo de Unit.
(Mínimo dos pruebas de unidad)



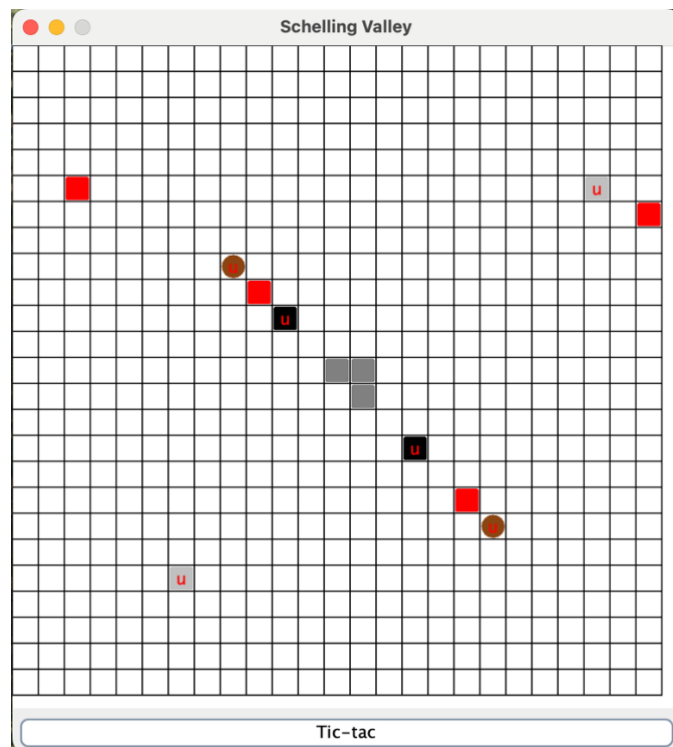
2. Considerando un par de ellos con el nombre de ustedes.
- (a) Piensen en otra prueba significativa y expliquen la intención.

Queremos asegurar que los árboles realmente actúan como obstáculos en el valle, impidiendo que los animales se muevan sobre ellos. Esta prueba sirve como prueba de aceptación, porque valida una regla de negocio esencial del juego: los obstáculos deben bloquear el paso.

- (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba.



- (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes



RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)

Juan Daniel Bogotá Fuentes 28 horas, más o menos 3 horas por día (jueves, lunes, martes, miércoles, jueves, viernes, sábado).

Nicolas Felipe Bernal Gallo 28 horas (Jueves[3 horas], lunes[2 horas], martes[3 horas], miércoles[1 hora], jueves[6 horas], viernes[6 horas], sábado[7 horas])

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

El estado actual del laboratorio es incompleto. Porque no pudimos realizar en su totalidad el laboratorio, profundizando en cada uno de los puntos y sus requerimientos.

3. Considerando las prácticas XP del laboratorio. ¿cuál fue la más útil? ¿por qué?

Dos programadores trabajan juntos en el código (Pair Programming). Esta es la práctica XP que nos fue más útil a la hora de realizar este laboratorio, puesto que nosotros nos intercambiábamos la labor de escribir código y revisar el código de la otra persona. También porque intercambiamos ideas y conocimiento que fuimos adquiriendo en el auto estudio de cada persona.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Entender todos los requerimientos que exigía el laboratorio a partir de las preguntas que fueron expuestas para su resolución. No dejando por nuestra parte lugar a la ambigüedad de nuestras respuestas.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico que presentamos a la hora de realizar este laboratorio fue la no interconectividad que presenta Blue J, donde no podíamos escribir en el mismo archivo al mismo tiempo, lo cual nos llevó a buscar otro entorno virtual que satisficiera esa necesidad.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Repartir de una buena forma el trabajo para que ambos aprendamos y además tengamos la misma carga de trabajo. Nos comprometemos a repartir mejor las horas por día para hacer el laboratorio

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

ChatGPT. (s/f). Chatgpt.com. Recuperado el 14 de octubre de 2025, de <https://chatgpt.com/>
Rodríguez, A. (s. f.). Análisis y casos. Grafos - software para la construcción, edición y análisis de grafos. Recuperado el 14 de octubre de 2025, de <https://arodrigu.webs.upv.es/grafos/doku.php?id=analisis>

Vypirailenko, A. (2023, julio 21). División entera Java. CodeGym. <https://codegym.cc/es/groups/posts/es.696.division-entera-java>